

Docker Compose Handson

Docker Compose is a tool that lets you define and run **multi-container Docker applications** using a single YAML file (`docker-compose.yml`). Instead of running multiple docker run commands manually, Compose allows you to define your services (containers), volumes, networks, etc and start/stop them all together with one command: `docker-compose up/down`. It helps easily manage multi-container apps (e.g., app + database). Infrastructure-as-code: store everything in a `docker-compose.yml` file. Consistency across teams/environments due to reproducible development & testing environments.

Supported filenames: `docker-compose.yml`, `docker-compose.yaml`, `compose.yml`, `compose.yaml`

1. Check and install `docker-compose` on local machine :

\$ `docker-compose --version` # To check if docker-compose is installed on lab machine.

\$ `sudo apt install docker-compose` # To install the docker-compose on lab machine.

```
(base) labuser@ip-172-31-9-72:~$ docker-compose --version
Command 'docker-compose' not found, but can be installed with:
snap install docker          # version 27.5.1, or
apt install docker-compose   # version 1.29.2-1
See 'snap info docker' for additional versions.
(base) labuser@ip-172-31-9-72:~$
(base) labuser@ip-172-31-9-72:~$ _
```

2. Create a sample `docker-compose.yaml` file and custom `index.html` to host on nginx web server.

```
(base) labuser@ip-172-31-9-72:~$ cat > docker-compose.yaml

version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    volumes:
      - ./index.html:/usr/share/nginx/html/index.html
  db:
    image: postgres:13
    environment:
      POSTGRES_DB: mydatabase
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

```
services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    volumes:
      - ./index.html:/usr/share/nginx/html/index.html
  db:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydatabase
      POSTGRES_USER: user
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

Note : See the volumes section where we are defining a named volume `db_data` to be created. To make it unique across different docker compose projects, docker-compose will prefix the project name (provided during the docker compose command) or current-directory-name. So our volume name will be created as `labuser_db_data` (since our current directory name is labuser).

Create a custom `index.html` file as well to host it on nginx webserver, port 8081

```
(base) labuser@ip-172-31-9-72:~$ cat > index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx via Docker Compose!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1> Welcome to nginx via Docker Compose!</h1>
<p>If you see this page, the nginx web server is successfully installed and
Working via Docker Compose. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx via Docker Compose!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1> welcome to nginx via Docker Compose!</h1>
<p>If you see this page, the nginx web server is successfully installed and
Working via Docker Compose. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

3. Deploy the containers mentioned in `docker-compose.yaml` file : `$ docker-compose up -d`

“-d” starts the containers in detached mode, if not used ,it will display live logs inside the containers.

```
(base) labuser@ip-172-31-9-72:~$ docker-compose up -d
Creating network "labuser_default" with the default driver
Creating labuser_db_1 ... done
Creating labuser_web_1 ... done
(base) labuser@ip-172-31-9-72:~$ _
```

4. List containers launched by Docker-compose and details, notice the prefix `labuser_` due to project/dir name:

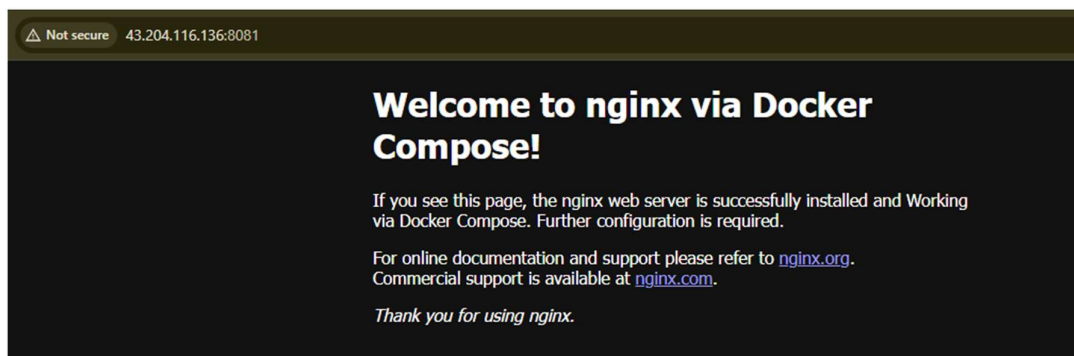
```
(base) labuser@ip-172-31-9-72:~$ docker-compose ps
```

Name	Command	State	Ports
labuser_db_1	docker-entrypoint.sh postgres	Up	5432/tcp
labuser_web_1	/docker-entrypoint.sh nginx ...	Up	0.0.0.0:8081->80/tcp, :::8081->80/tcp

```
(base) labuser@ip-172-31-9-72:~$
```

5. Since we hosted our web container/service on 8081, lets check the `<<LAB-IP>:8081` if config worked or not :

`http://43.204.116.136:8081/`



6. Let's check the logs inside our web/db containers :

\$ **docker-compose logs** # Will display logs collected till now and quit.

\$ **docker-compose logs -f** # Will display logs collected till now and continue to stream unless quit.

```
(base) labuser@ip-172-31-9-72:~$ docker-compose logs -f
Attaching to labuser_web_1, labuser_db_1
db_1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db_1 |
db_1 | 2025-08-05 06:38:01.207 UTC [1] LOG: starting PostgreSQL 13.21 (Debian 13.21-1.pgdg120+1) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 12.2.0-14) 12.2.0, 64-bit
db_1 | 2025-08-05 06:38:01.208 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db_1 | 2025-08-05 06:38:01.208 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db_1 | 2025-08-05 06:38:01.217 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db_1 | 2025-08-05 06:38:01.232 UTC [27] LOG: database system was shut down at 2025-08-05 06:37:16 UTC
db_1 | 2025-08-05 06:38:01.241 UTC [1] LOG: database system is ready to accept connections
web_1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
web_1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web_1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
web_1 | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
web_1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
web_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
web_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
web_1 | /docker-entrypoint.sh: Configuration complete; ready for start up
web_1 | 2025/08/05 06:38:01 [notice] 1#1: using the "epoll" event method
web_1 | 2025/08/05 06:38:01 [notice] 1#1: nginx/1.29.0
web_1 | 2025/08/05 06:38:01 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14+deb12u1)
web_1 | 2025/08/05 06:38:01 [notice] 1#1: OS: Linux 6.5.0-1017-aws
web_1 | 2025/08/05 06:38:01 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
web_1 | 2025/08/05 06:38:01 [notice] 1#1: start worker processes
web_1 | 2025/08/05 06:38:01 [notice] 1#1: start worker process 28
web_1 | 2025/08/05 06:38:01 [notice] 1#1: start worker process 29
web_1 | 2025/08/05 06:38:01 [notice] 1#1: start worker process 30
web_1 | 2025/08/05 06:38:01 [notice] 1#1: start worker process 31
web_1 | 119.252.206.169 - - [05/Aug/2025:06:40:09 +0000] "GET / HTTP/1.1" 200 674 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0"
web_1 | 2025/08/05 06:40:09 [error] 28#28: *2 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 119.252.206.169, server: localhost, request: "GE"
web_1 | 119.252.206.169 - - [05/Aug/2025:06:40:09 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://43.204.116.136:8081/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
web_1 | 119.252.206.169 - - [05/Aug/2025:06:44:36 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 S"
web_1 | 119.252.206.169 - - [05/Aug/2025:06:45:04 +0000] "\x16\x03\x01\x06\xC0\x01\x00\x06\x8C\x03\x03\xA3|\xB9?\xA9\x875\xDA\xB00\x898\xCE\xCB\xAFA\xC2h" 400 157 "-" "-" "-"
web_1 | 119.252.206.169 - - [05/Aug/2025:06:45:04 +0000] "\x16\x03\x01\x07\x00\x01\x00\x06\xFC\x03\x03\x09\xA5\xB6\xF6\xD6? \xB8 \xF3)\x17\x1B\xE8F\xE4a\xD0a02\x98\x84\xA3\xECf\x9"
```

7. We can also scale the containers individually if needed : \$ **docker-compose up --scale db=2**

8. Time to shut down the services : **docker-compose down**

Check the volume and we still see volume not removed even though docker compose down executed.

```
(base) labuser@ip-172-31-9-72:~$ docker-compose down
Removing labuser_db_2 ... done
Removing labuser_web_1 ... done
Removing labuser_db_1 ... done
Removing network labuser_default
(base) labuser@ip-172-31-9-72:~$
(base) labuser@ip-172-31-9-72:~$ docker volume ls
DRIVER      VOLUME NAME
local       labuser_db_data
```

\$ **docker volume ls**

```
(base) labuser@ip-172-31-9-72:~$ docker volume ls
DRIVER      VOLUME NAME
local       labuser_db_data
(base) labuser@ip-172-31-9-72:~$
```

9. If we need to remove the volume (when hands-on complete) we can do it via **docker-compose down -v** :

Let's bring up the services again using : **docker-compose up -d**

And this time use different command to remove volume as well : **docker-compose down -v**

```
(base) labuser@ip-172-31-9-72:~$ docker-compose up -d
Starting labuser_web_1 ... done
Starting labuser_db_1 ... done
(base) labuser@ip-172-31-9-72:~$
(base) labuser@ip-172-31-9-72:~$ docker-compose down -v
Stopping labuser_web_1 ... done
Stopping labuser_db_1 ... done
Removing labuser_web_1 ... done
Removing labuser_db_1 ... done
Removing network labuser_default
Removing volume labuser_db_data
(base) labuser@ip-172-31-9-72:~$ docker volume ls
DRIVER      VOLUME NAME
(base) labuser@ip-172-31-9-72:~$
```

So now both web/app containers (and all replicas) along with docker volume is removed.